Churn Prevention Agent

How the project is structured, and where we are now

Churn here means a customer closed their credit card. They remain a bank customer — they ended one product.

1

Setup and planning

DONE

Decide what we are building and why, choose the tools, set up the project folder, GitHub and the question log for the manager.

Produces: Project brief, plan, setup guide, questions log

2

Data analysis and feature engineering

IN PROGRESS

Understand the file. Check it for problems that would quietly ruin the model. Compare the two groups of customers, test ideas, and build the new columns the model will learn from.

Produces: A clean, documented dataset and every decision written down

3

The model

NOT STARTED

Split the data, build a simple baseline to beat, train the real model, judge it honestly, and make it explain each individual prediction.

Produces: A saved model that returns a prediction, a probability and its reasons

4

The agent

NOT STARTED

Write the offer catalogue, the tool-use loop and the chat interface. The agent collects the customer's details, calls the model, reads the reasons and proposes a fitting offer.

Produces: A working assistant the marketing team can talk to

5

Testing and write-up

NOT STARTED

Run real customers through the whole system, record what happens, then write the report and build the presentation.

Produces: Report, slides and a rehearsed live demo

Rough share of the work

My estimate, not a measured fact. Phases 2 and 3 together are the bulk of it.

Phase 2 — Data analysis and feature engineering

The order we work in, and why each step comes where it does

1. **Load it and check its condition**

   How many rows and columns, what types, anything missing, anything duplicated. Five minutes that tells you whether the rest is worth doing.

2. **Look at the answer column**

   How lopsided is it? This one number decides which measures we are allowed to judge the model on. With one in five closing their card, accuracy becomes useless.

3. **Check for leakage**

   Does any column know something it only could have known afterwards? This is the mistake that quietly ruins projects, because the model looks excellent while being worthless. Method: assume the worst case, work out what the data would look like if it were true, then check.

4. **Check for overlap**

   Are two columns saying the same thing? Not cheating, but it splits the model's reasoning across one duplicated idea and makes explanations repeat themselves.

5. **Compare the two groups, one column at a time**

   The whole job in one line: find what is different about the customers who closed their card. A column is only useful if the two groups behave differently on it.

6. **Look at pairs of columns**

   Two factors can each mean little alone and a great deal together. These combinations are where a tree model earns its keep.

7. **Form ideas, then test them honestly**

   Statistics cannot invent a hypothesis, only test one. A person has to supply the idea. Most will fail — record those too, or you are only showing yourself the flattering half.

8. **Build the new columns**

   Turn what you learned into columns the model can read. This is feature engineering: not new data, just making visible what was already there.

9. **Check each new column earns its place**

   Building a column is not the same as it being useful. Score every one. Anything that separates nobody needs a reason to stay.

10. **Check the new columns are not copies of each other**

    Two columns carrying the same information muddy both the model and the explanation.

11. **Save it, and write down every decision**

    The processed dataset for the next phase, plus the reasons behind each choice. A decision with no reason is not a decision yet.

Phase 3 — The model

What comes next, in order

1. **Split the data first, before anything else**

   Hold back a portion of customers and do not look at them again until the end. If any information from them reaches training, the score we report is fiction.

2. **Build a deliberately simple baseline**

   A basic model, so there is a number to beat. Without one there is no way to say whether the complicated model was worth using — and if the simple one comes close, that is a real finding.

3. **Train the real model**

   Gradient boosted trees, because the data is a table of mixed types and the effects are combinations rather than straight lines.

4. **Judge it honestly**

   Never accuracy. Of the customers we flagged, how many really closed their card? Of those who closed, how many did we catch? Those are the questions the marketing team cares about.

5. **Choose the cut-off deliberately**

   Where we draw the line between 'will close' and 'will not' is a business decision, not a default. It depends on whether missing someone costs more than wasting an offer — a question for the manager.

6. **Make it explain itself**

   For each individual customer, which fields pushed the answer toward closing. This is what the agent reads to build a personal offer, and without it the agent would be inventing reasons.

7. **Connect the feature names to plain language**

   The model talks about column names. The offer catalogue talks about customer problems. Something has to translate, and this step is where the two halves of the project join.

8. **Save the model and swap it in**

   Replace the placeholder inside the prediction function. The tests written earlier must still pass, unchanged. That is the proof nothing downstream broke.

How everything stays connected

Google Colab Where the notebooks run. No local setup to fight with, and reachable from any machine.

GitHub The single source of truth. Colab saves notebooks straight to it (File, then Save a copy in GitHub), and the docs, code and charts live there too.

The docs folder Every decision and its reason. The notebooks show what was done; the docs say why.

data/processed The handover between phases. Phase 2 writes the prepared dataset, phase 3 reads it and never repeats the preparation.